

Lab 6: ARP & Local Network Exploration

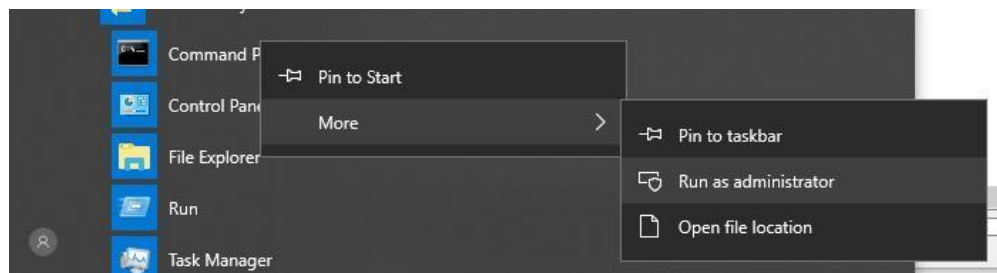
In this lab, we'll use a combination of Windows tools, plus *Wireshark*, to observe the *Address Resolution Protocol (ARP)*, discover information about our local network and exactly *who* we are connected to, and then set up a *static ARP entry* for our computer to help prevent possible *man-in-the-middle* attacks via *ARP spoofing*.

Note that this lab *assumes you are using a wireless (Wi-Fi) connection under Windows*. It is possible to do this lab with a wired (Ethernet) connection, although you will need to *look at a different connection interface*. If you are on *Mac or Linux*, you can still take part in this lab, however the commands required will be different. If you need assistance, feel free to ask – we should be able to work around it.

This is primarily a learning experience and helping secure your own home network connection – *the only item you need to submit is a screenshot showing the static ARP cache entry that you will create in this lab*.

Checking Our IP Configuration

To start, we're going to need to open the *Windows Command Prompt* in *Administrator Mode*, which can be found under *Windows System* in your Start Menu (on Windows 10). Right-click and select "*Run as Administrator*".



Once you've got that open, we'll run a command to give us a *summary* of our network connections known as *ipconfig*. Ipconfig is a very useful utility (equivalent in Mac/Linux is *ifconfig*) that returns the status, addresses, names and other general information on all the *network interfaces* (NICs) that our computer is using. Try running the *ipconfig* command.

The output will appear as a set of tables, with each table describing *one* of the network interfaces. You may see various interfaces depending on your own setup, including *Bluetooth*, *Ethernet* interfaces for wired connections, *Wireless LAN adapters* (if you've got a Wi-Fi connection or adapter), as well as possibly *local loopback* and other local network interfaces used internally in your machine. Look through this output and try to find the connection you are using for your internet. It should have an *IPv4* (or possibly *IPv6*) address associated with it, and it should also have a *Default Gateway*, which is typically your home *router or modem*.

```

Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix  . : okanagan.bc.ca
Link-local IPv6 Address . . . . . : fe80::f900:76c0:c1ee:8df8%9
IPv4 Address. . . . . : 10.1.165.202
Subnet Mask . . . . . : 255.255.240.0
Default Gateway . . . . . : 10.1.160.1

```

In this case (at Okanagan College on the OKC Wi-Fi), it looks like my *default gateway* is **10.1.160.1**. This is a *local IP address* (local IP addresses typically begin with 10, 192.168, or 172.16) which is the IP address of the router, firewall, or other network device that your computer's internet traffic is sent through. Depending on the setup of your network, you may be able to visit this IP address in your *web browser* to find the *router configuration page*, although this isn't necessary. Some routers may also host their configuration pages at a different local IP address.

Viewing the Local ARP Cache

So, we know what IP address we are connected to, but let's learn some more information about our local network. Try typing in the command *arp -a* in your command prompt (sudo arp -a is used in Linux/Mac). This should print out your local *ARP cache*, which as we discussed, is a local copy of *IP address to MAC address mappings* that your machine knows. This should print out something that looks like the following:

```

C:\Windows\system32>arp -a

Interface: 10.1.165.202 --- 0x9

Internet Address      Physical Address      Type
10.1.160.1            b4-0c-25-e2-80-47    dynamic
10.1.166.75           b8-a1-75-4d-43-0f    dynamic
10.1.175.255          ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static

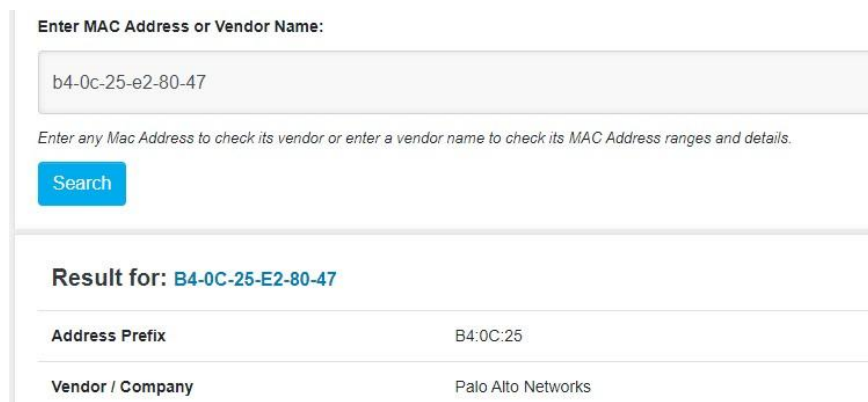
```

You will notice that each *mapping* is also specified as being either *static* or *dynamic*. Dynamic ARP cache entries can be *updated* or refreshed using the *ARP* protocol that we discussed in class. In particular, pay attention to the entry that matches the IP address of our *default gateway* that we saw earlier. Make a note of the fact that it is a *dynamic entry*, and also the *physical address* (aka the MAC address) of the device. Try checking your home router/modem for a sticker (usually on the back) that lists the MAC address of the device. Does it match?

Next, try going to the following website:

<https://dnschecker.org/mac-lookup.php>

This tool allows you to enter your **MAC address** and find out some information about the **vendor** that manufactured it. Note that this is typically the vendor of the Network Interface Controller (NIC) **chipset**, who is not necessarily the same as the **brand** of the router. For example, you may find that your **Telus**, **Rogers**, or **D-Link** brand router returns a vendor like **Broadcom**, **Cisco**, **Intel**, or some other chip manufacturer. Try entering your **default gateway's MAC address** and see what vendor shows up.



Enter MAC Address or Vendor Name:

b4-0c-25-e2-80-47

Enter any Mac Address to check its vendor or enter a vendor name to check its MAC Address ranges and details.

Search

Result for: B4-0C-25-E2-80-47

Address Prefix	B4:0C:25
Vendor / Company	Palo Alto Networks

At the college, the local gateway is actually a **firewall**. Palo Alto Networks is a company known for producing **enterprise-grade** firewall hardware and given the college Wi-Fi is designed to handle many students and staff at once (some of whom may be *mischievous*), this isn't a big surprise.

Capturing ARP Traffic

Now, let's start up **WireShark** and try capturing some **ARP traffic**. To do this, we'll need to start recording, and then we'll **dump our ARP cache**. This will force your computer to perform ARP requests to re-establish the IP-to-MAC mappings that it previously held. **Once you have WireShark open and recording**, try running the **arp -d** command in the command prompt. You **must** be an administrator to do this. On Mac/Linux, you should be able to use **sudo arp -d**.

This will delete all entries in your ARP cache, generating new ARP requests and responses which will be captured by **WireShark**. Look for entries using the ARP protocol. You may wish to filter by "arp" to make them easier to find.

58	4.861484	IntelCor_dd:6a:a8	Broadcast	ARP	42	Who has 10.1.160.1? Tell 10.1.165.202
59	4.862882	PaloAlto_e2:80:47	IntelCor_dd:6a:a8	ARP	56	10.1.160.1 is at b4:0c:25:e2:80:47

```

▼ Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: IntelCor_dd:6a:a8 (c4:d9:87:dd:6a:a8)
  Sender IP address: 10.1.165.202
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 10.1.160.1

```

We can see that there are two ARP messages here: a **broadcast**, asking **who has** the default gateway IP address, and a **response** telling us who has it. First, let's look at the **broadcast request**.

Note that the *sender* has an IP address and a MAC address, but the *Target* only has an IP address. The target's MAC address is all zeroes (00:00:00:00:00:00), because this message is being sent to *all* other machines on the network.

To verify that it is *us* who is sending this broadcast, we can use `ipconfig /all` to print out a more complete list of network information, including the MAC address of *each interface on our machine*. See if you can find the *matching sender MAC address* in this information – it should be under your wireless (Wi-Fi) or wired (Ethernet) connection information. You should be able to find this information on *Mac/Linux* using `ifconfig` (without the `/all`).

```

▼ Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: PaloAlto_e2:80:47 (b4:0c:25:e2:80:47)
  Sender IP address: 10.1.160.1
  Target MAC address: IntelCor_dd:6a:a8 (c4:d9:87:dd:6a:a8)
  Target IP address: 10.1.165.202

```

Now, let's look at the response in WireShark. Note that the *sender* has a MAC address, which *should match the MAC address of your router or modem*. If an ARP spoofing/poisoning attack

was taking place, you'd find that the MAC address of the *reply sender* would be *different than your router's MAC address*, as they'd be trying to associate the *router's IP* with their own *MAC address* to redirect your traffic through their machine (where they could sniff it or modify it before passing it on to the real router).

If you found a sticker on your router/modem earlier that lists a MAC address on it, verify that this MAC address matches the MAC address of the sender, implying that no ARP spoofing is going on.

Preventing ARP Spoofing

Now, let's use a couple simple commands to help protect your machine from ARP spoofing attacks. Recall that your ARP cache entry for the default gateway (your router) most likely was in *dynamic mode*, which means that your machine may make *periodic ARP requests* to get the MAC address of your router. By changing this to a *static* entry, we can "lock in" the IP-to-MAC association for the router, making it impossible for an attacker to try to spoof the identity of your router (although they may still be able to spoof *your identity* to the *router*, if the router is sending ARP requests).

```

C:\Windows\system32>netsh interface ipv4 show interfaces

```

Idx	Met	MTU	State	Name
1	75	4294967295	connected	Loopback Pseudo-Interface 1
9	40	1500	connected	Wi-Fi
12	65	1500	disconnected	Bluetooth Network Connection
3	25	1500	disconnected	Local Area Connection* 2
13	5	1500	disconnected	Ethernet

First, we'll need to *list the interfaces* on your machine to find the *name* of the interface that we are using for our connection. This can be done using the *netsh interface ipv4 show interfaces* command, which will list all your interfaces. Look for a *connected* interface that will be either a *Local Area Connection* (if wired), or possibly a *Wi-Fi* connection (if wireless).

Next, we'll use the *netsh interface ipv4 set neighbors* command to add a *static ARP cache entry* for our default gateway (the router). You'll need to pass it the *name* of the interface in quotes (here I'm using "Wi-Fi"), the *IP address of the default gateway* (10.1.160.1 in this example), and the *MAC address* of the router (b4-0c-25-e2-80-47 in this example). This will insert a new static ARP entry for the Wi-Fi interface that maps the gateway/router IP with the router MAC address, without requiring further ARP requests down the line. On Mac and Linux, you should just be able to use *arp -s <ip> <mac>* to create a static mapping of IP to Mac address.

```
C:\Windows\system32>netsh interface ipv4 set neighbors "Wi-Fi" 10.1.160.1 b4-0c-25-e2-80-47
```

Finally, we can *verify* that this has worked by checking our *ARP cache* again using *arp -a*. Now, we should see that the default gateway entry is of the *static type*, rather than the *dynamic type*. Congratulations, you've helped protect yourself against ARP spoofing.

```
C:\Windows\system32>arp -a

Interface: 10.1.165.202 --- 0x9
    Internet Address      Physical Address      Type
    10.1.160.1            b4-0c-25-e2-80-47    static
```

*Once you've gotten to this point, take a screenshot that shows your *ipv4 set neighbours* command as well as the output of your *ARP cache* showing the new static record.*

Lab Submission

When finished, submit your screenshot of your ARP adjustments on the course Moodle. Please submit your screenshot in JPG, PNG, or WEBP image format please – no PDFs or Word documents.